

JPSME 2.0 Blueprint

A drawing set for how the Junior Philippine Society of Mechanical Engineers' membership platform actually moves people, invitations, and money through the system — from a name typed into a registration form to a peso landing in the org's PayMongo balance.

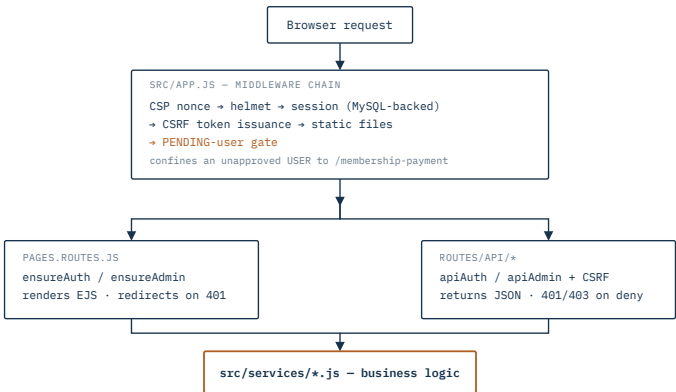
STACK Express · Prisma · MySQL · EJS	DRAWN FROM CLAUDE.md, this repo	SHEETS 00 - 05	SCALE mechanism, not file tree
---	------------------------------------	-------------------	-----------------------------------

SHEET 00

System Overview

Every request enters through one middleware stack, then splits into two parallel route trees that both bottom out in the same service layer. The split — and the gate sitting in the middle of it — is the piece that isn't obvious from any single file.

FIG 00.1 — REQUEST PIPELINE



Both route trees are thin — controllers glue req/res to a service call, and every service talks to MySQL only through Prisma.

THE GATE

A **PENDING** user gets a real session (to pay) but every path except `/membership-payment`, `/logout`, and `/api/payments` bounces them back.

ADMIN PANEL QUIRK

Admin pages are also fetched as fragments (`X-Requested-With: fragment`) for the sidebar's SPA-style navigation.

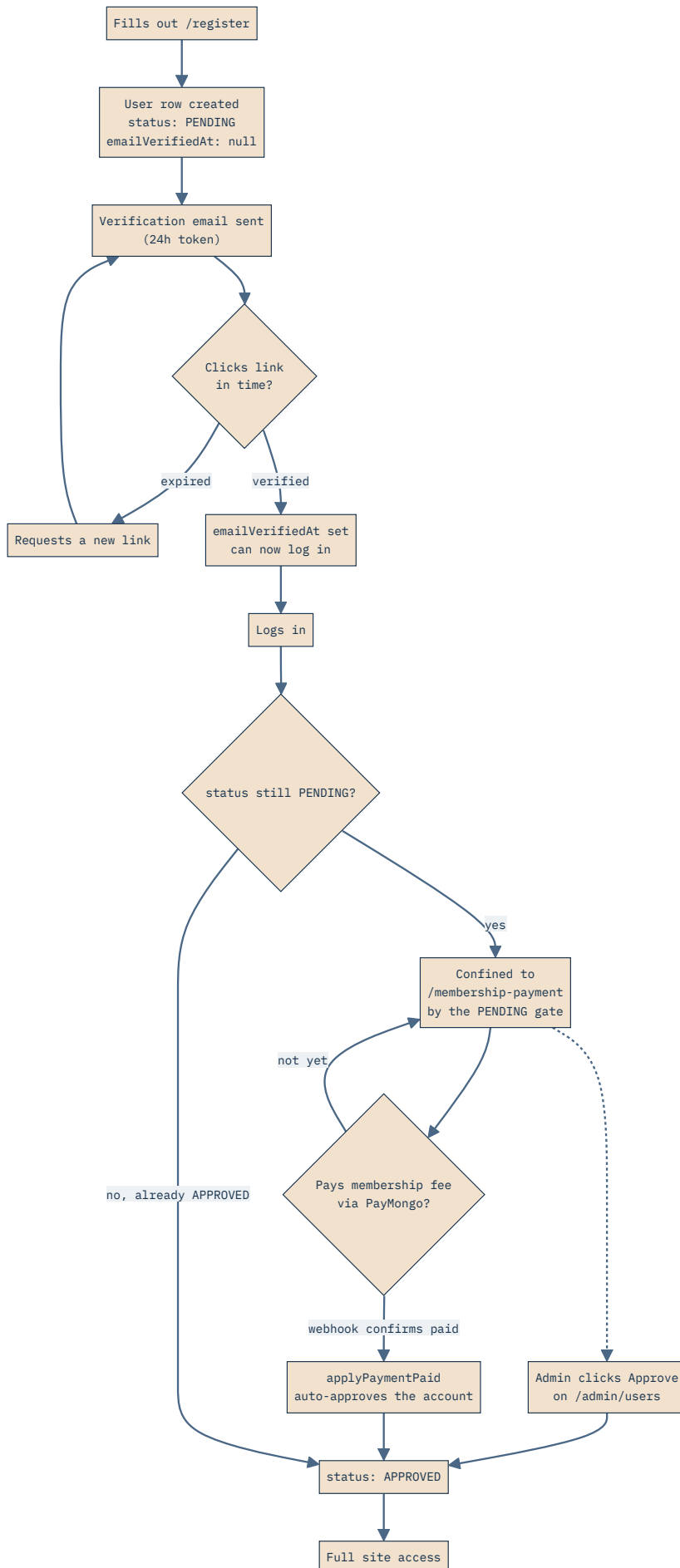
ROLES

AREA	ADMIN	CHAPTER_A
Payments, invitations, settings	full	—
Member/chapter management	full	own chapter
Event registration (as a user)	blocked	blocked

Membership Lifecycle

Registering an account and becoming an approved member are two separate gates — email ownership, then a human (or a payment) saying yes.

FIG 01.1 – REGISTER TO APPROVED



SAME TRAIL EITHER WAY

Auto-approve calls the identical `userService.setStatus(...)` the manual button uses — same audit log, same approval email.

IF IT FAILS SILENTLY

A thrown error inside auto-approve is caught and logged as an

- **AUTO_APPROVAL_FAILED** audit entry — check the Audit Log page.

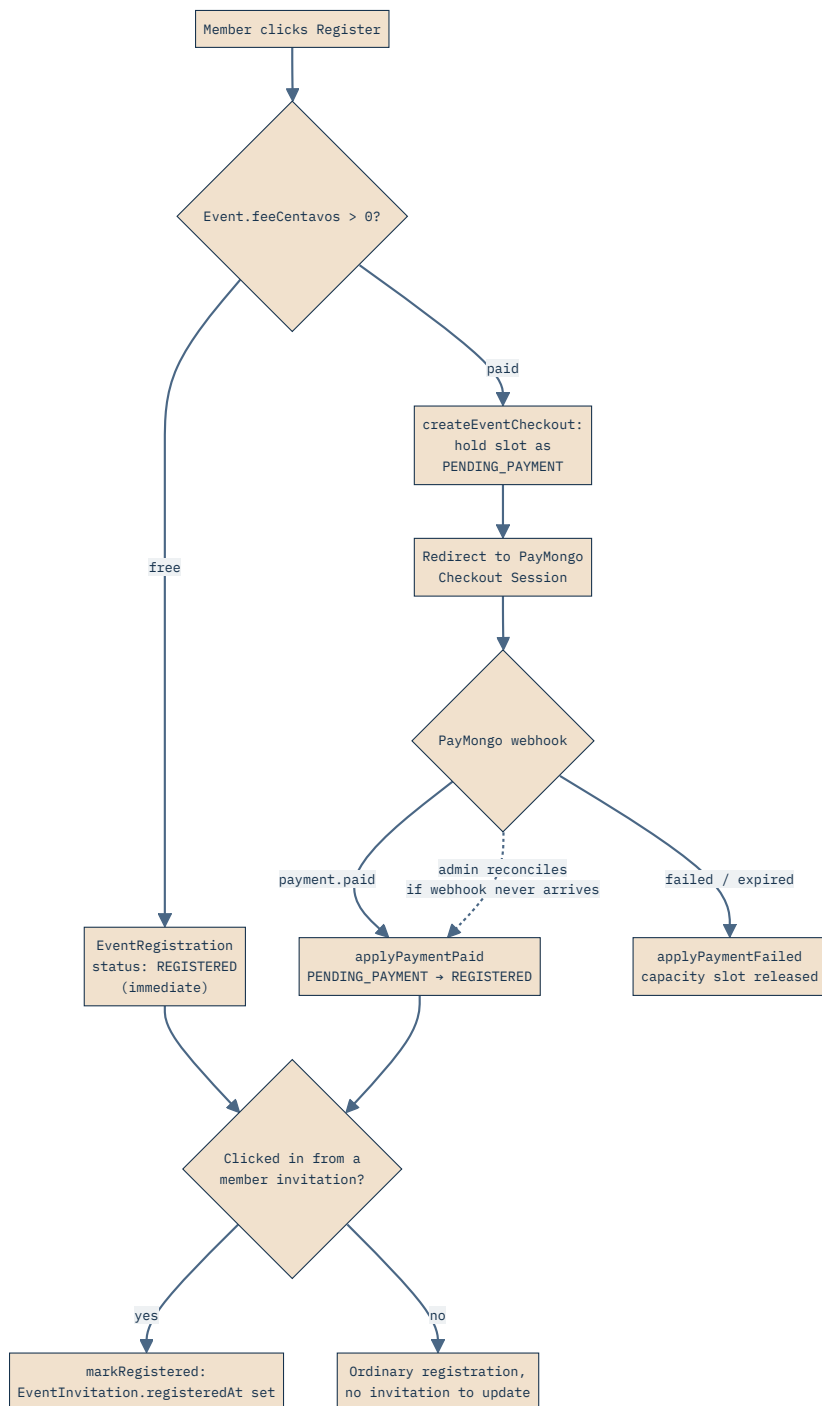
ONE-TIME REDIRECT

Registering from an invitation link stores `postApprovalRedirectUrl` — the first login after approval lands back on that event.

Event Registration – Free vs. Paid

A free event resolves in one write. A paid one holds the capacity slot before money ever moves, and only a verified webhook confirms it.

FIG 02.1 – REGISTRATION BRANCH



NEVER TRUST THE REDIRECT

The payment-return page only ever reads current DB state — it cannot itself mark anything paid.

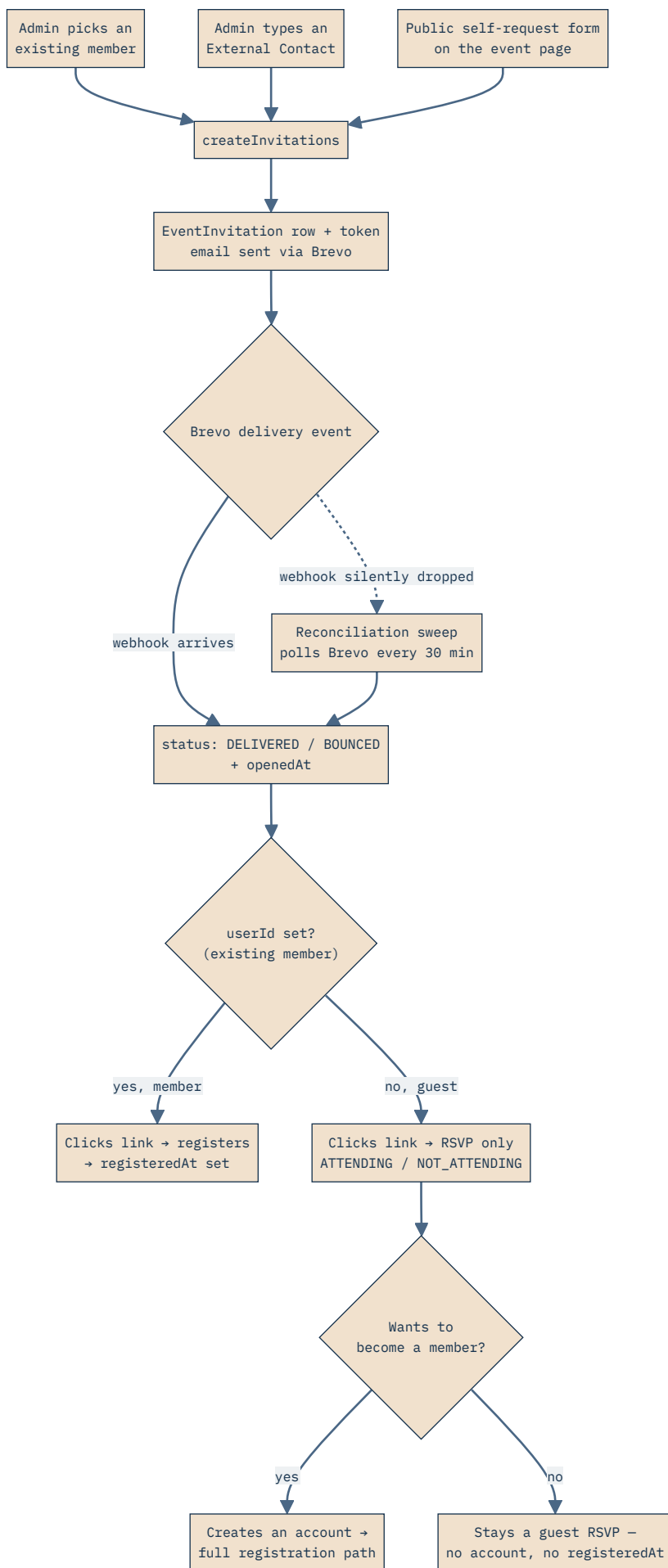
THE FEE, EXACTLY

The member is charged the event fee plus PayMongo's own cut as a second, itemized line — see Sheet 04.

Invitation System

Three different doors create the same kind of row. What happens after sending then forks hard on one question: does this person already have an account?

FIG 03.1 – INVITE TO OUTCOME



TWO DIFFERENT SIGNALS

`registeredAt` means a real registration.
`rsvpStatus` is guest-only and never implies one.

RECONCILIATION IS PROVEN, NOT ASSUMED

Found by cross-checking Brevo's own Statistics API against this app's DB.

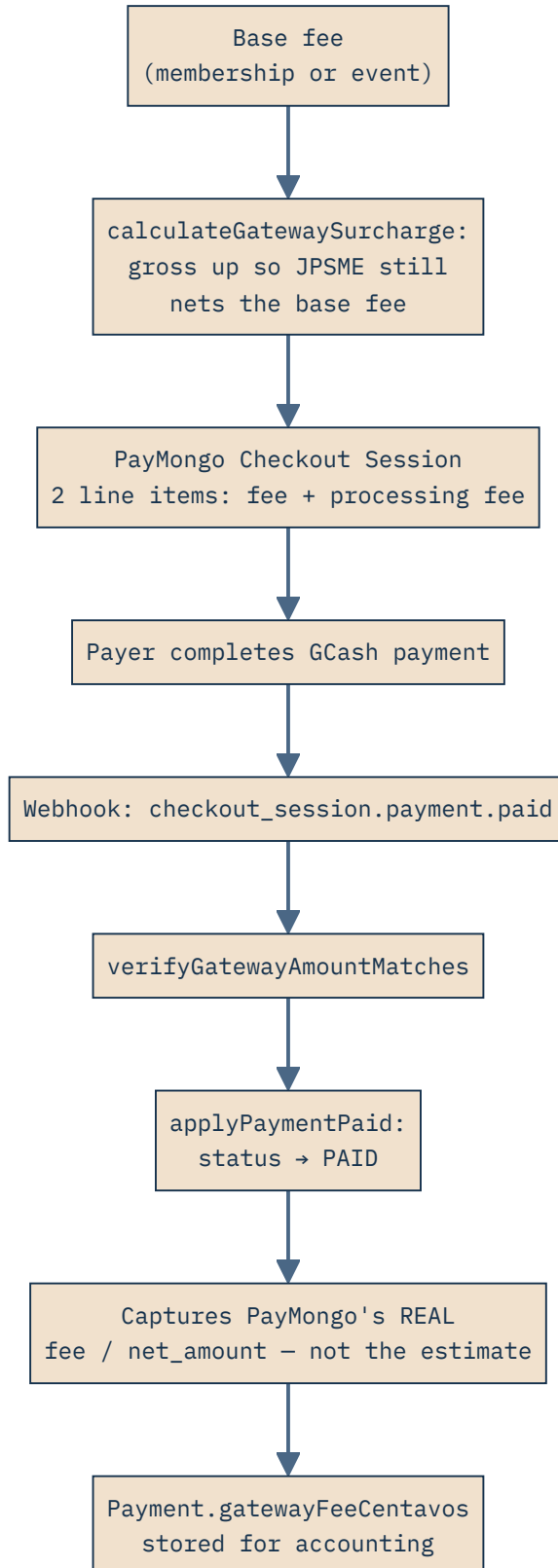
NEEDS A LIVE TUNNEL

Brevo's webhook needs an externally reachable URL (ngrok in development).

Payments & Fee Accounting

PayMongo hosts the actual payment page. The part worth drawing precisely is the gap between what gets charged up front and what PayMongo actually reports afterward.

FIG 04.1 – CHECKOUT TO RECONCILED LEDGER



ONE CHOKE POINT

`applyPaymentPaid` / `applyPaymentFailed` are shared by the live webhook and admin-triggered reconciliation.

NO SAFETY BUFFER, BY DESIGN

The gross-up formula is exact, not padded – see fig 04.2.

FIG 04.2 – ESTIMATE VS. ACTUAL

CHARGED AT CHECKOUT (estimate)

Base fee ₱500.00	₱12.81
------------------	--------

PAYMONGO'S REAL DEDUCTION (settlement)

Net received ₱500.00	₱12.82
----------------------	--------

+ 1¢ rounding gap
two formulas, computed independently –
expected, recorded, never hidden

Both numbers are real: the left bar is what the payer was charged; the right is what PayMongo's settlement API reports after the fact.

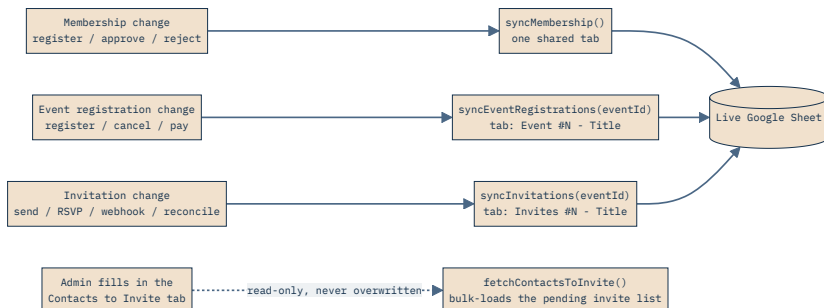
WHY IT DRIFTS

PayMongo's own fee math applies its percentage, then VAT, then rounds – this app's estimate does it in one step.

Live Google Sheets Sync

One tab per event, for both registrations and invitations — not one tab for everything, which is exactly the shape that floods as the org grows.

FIG 05.1 — TRIGGERS TO TABS



FULL REWRITE, NOT A PATCH

Every sync clears and rewrites its one tab rather than editing individual cells.

FIRE-AND-FORGET

Sync calls never block the action that triggered them and swallow their own errors.

STABLE TAB NAMES

Tabs are matched by an `Event #<id>` / `Invites #<id>` prefix, not the full title.

Drawn from the codebase itself — see CLAUDE.md for the prose version of every sheet here.

JPSME 2.0 · Express + Prisma + MySQL